

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)

Assessment Tool Weightage: 30%

Dr Dumpala Prasanth

QUESTIONS: 10

Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Duration: 2Hrs

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.
(10 Marks)
2. Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.
(10 Marks)
3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.
(10 Marks)
4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.
(10 Marks)

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design. (10 Marks)
6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint. (10 Marks)
7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization. (10 Marks)
8. Analyse an experimental comparison between random action selection and ϵ -greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance. (10 Marks)
9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process. (10 Marks)
10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making. (10 Marks)
-

Code of Conduct:

(192425095)

I Ch. Harsha Chakravarthy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

CO1 - AT2

1. Grid World Navigation using OpenAI Gym

State Space: Every cell in a grid represents a state (row, column).

Action Space:

- up
- Down
- Left
- Right

Reward Function:

- Goal Reached = +100
- Normal move = -1
- Hit obstacle = -20
- Invalid move = -5

Constraints:

- cannot move outside the grid.
- Normal move = -1
- Hit obstacle = -20
- Invalid move = -5

Python code:

```
import gym
```

```
states = [(i, j) for i in range(5) for j in range(4)]
```

```
actions = ["up", "down", "left", "right"]
```

```
reward = {
```

```
    "goal": 100,
```

```
    "move": -1,
```

```
    "obstacle": -20
```

```
}
```


2. ϵ -Greedy Multi-Armed Bandit

Algorithm

1. Initialize rewards.
2. With probability ϵ explore.
3. Otherwise exploit the best arm.
4. Repeat for 500 trials.

Python code:

epsilon = 0.1

trials = 500

Comparison:

ϵ	Exploration	Exploitation	Performance.
0.1	Low	High	Highest reward
0.3	Balanced	Balanced	Good
0.5	High	Low	More exploration, Lower reward.

Conclusion: $\epsilon = 0.1$ gives the highest cumulative reward after 500 trials.

3. MDP for Autonomous Robot:

States

- Robot positions
- Battery level

Actions

- Move North
- Move South
- Move East
- Move West

Transition Probability:

- Correct move = 0.9
- Wrong move = 0.1

Rewards

- Reach destination = +100
- Every move = -2
- Battery exhausted = -100

Energy Constraint.

Battery consumption must remain below the available energy.

4. Value Iteration in Constrained Grid

Python

for iteration in range(100):

$$V[s] = \max(\text{sum}(p * (R + \text{gamma} * V[\text{next}]) \text{ for next in states}))$$

Bellman Equation

$$V(s) = \max_a \sum_{s'} P(s'|s, a) [R + \gamma V(s')]$$

Restricted states are skipped during updates.

Result:

- Finds the optimal policy.
- Avoids restricted cells.
- Maximizes long-term reward.

5. Warehouse Robot with Battery Limit:

Constraint:

Battery capacity = 30 minutes.

State:

- Robot position
- Remaining battery.
- Delivery status.

Policy:

Select the nearest delivery that can be completed with remaining battery.

Rewards:

- Successful delivery = +50
- Battery saved = +5
- Battery exhausted = -100

Goal: complete maximum deliveries before battery ends.

6.

Traffic Signal RL with Emergency Vehicles

State:

- Queue length
- Vehicle count
- Emergency vehicle presence

Actions:

- Green - North-south
- Green East-west

Reward:

- Reduced waiting times = +10
- Emergency vehicle passes = +100
- Congestion = -20

Safety Constraint:

Whenever an emergency vehicle is detected, exploration is disabled and the signal immediately turns green for that lane.

7. Wireless packet Routing:

Python

```
import tensorflow as tf
```

```
model = tf.keras.Sequential([
```

```
    tf.keras.layers.Dense(32, activation='relu'),
```

```
    tf.keras.layers.Dense(4)
```

])

States

- Network congestion
- Queue length

Actions:

- Select routing path

Bandwidth constraint

if bandwidth usage exceeds the limit:

Reward = -20

otherwise:

Reward = +10

Policy learns routes with minimum congestion.

8. Random vs ϵ -Greedy selection

Method	Cumulative Reward	Performance
Random	Low	Poor
ϵ -Greedy	High	Better

Random keeps exploring

ϵ -Greedy gradually exploits the best action higher cumulative rewards

within same time limit.

9. RL Framework for Delivery Drone

MDP

States

• Drone location

• Battery

• Weather

Actions:

Fly North

South

East

West

Return

Rewards:

• Successful delivery = +100

• Battery loss = -5

• Enter no-fly zone = -100

Bellman Equation

$$V(s) = \max_a [R + \gamma V(s')]]$$

Policy learns safe routes while avoiding restricted zones and preserving battery.

Smart Energy Management System:

constraint

Electricity consumption must remain below the threshold.

Reward Shaping

- stay below limit = +20

- save energy = +10

- Exceed threshold = -50

Policy Learning

The RL agent learns appliance scheduling to minimize power usage while maintaining performance.

feasibility constraints

- Maximum power limit

- Peak-hour restrictions

- Device priority

Outcome:

The system reduces consumption while maintaining efficient operation.

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided